

garganttua-api-example — Benchmark des modes de réflexion

runtime vs aot vs native — 20 exécutions par mode

Date : 2026-06-01 18:34
CPU : Intel(R) Core(TM) Ultra 7 165U (14 coeurs)
RAM / OS : 30.5 GiB / Linux 6.17.0-29-generic
JDK (jars) : java version "21.0.5" 2024-10-15 LTS
Build natif : native-image 21.0.5 2024-10-15

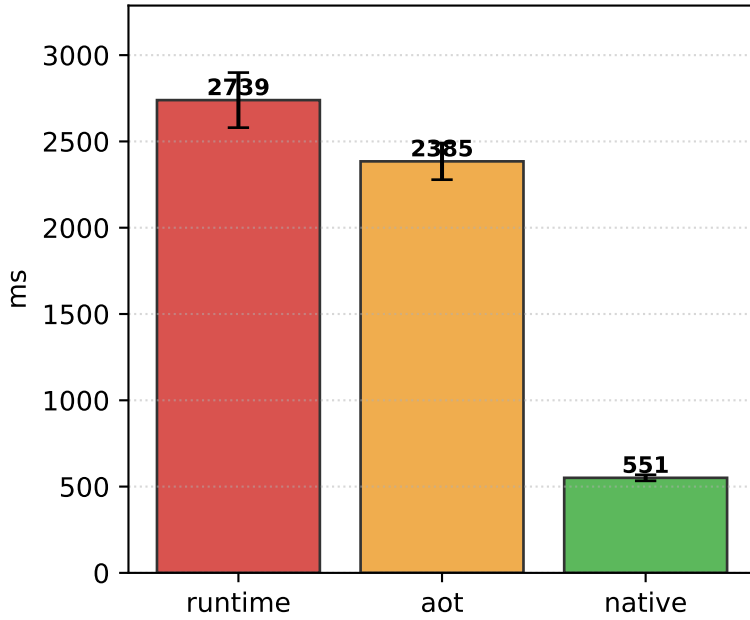
Methodo : 3 runs de chauffe ecartes, puis 20 runs mesures par mode. Meme charge dans tous les modes. Temps mur = duree du process complet ; CPU = temps CPU user+sys (ru_*time) ; RSS pic = ru_maxrss ; startup = bootstrap rapporte par l'app. os.wait4, runs sequentiels, meme machine.

mode	temps mur ms (moy±σ)	startup (ms)	RSS pic (MB)	CPU (ms)	CPU (%)	exécutable (MB)
runtime	2739 ± 160	1256	452	7031	257 %	6
aot	2385 ± 106	894	446	6124	257 %	5
native	551 ± 18	231	138	549	100 %	37

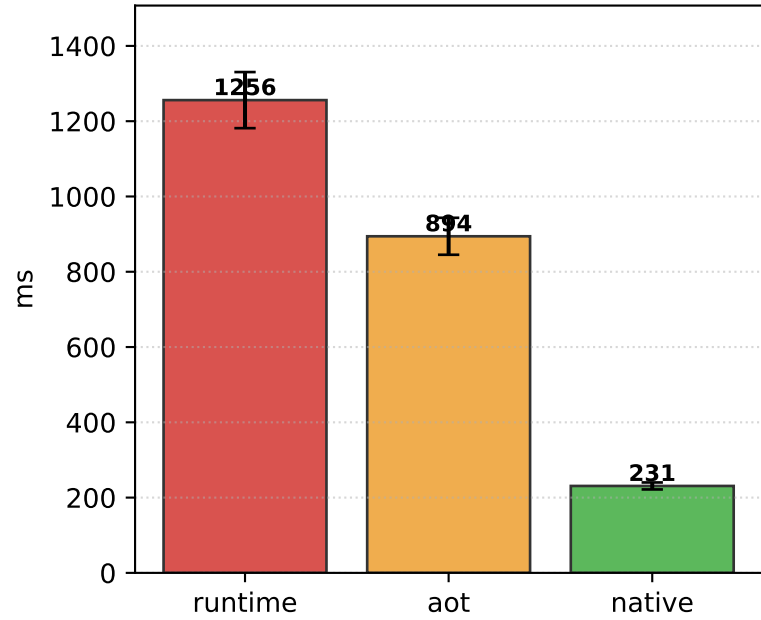
- Synthèse
- native : 5.0x plus rapide (temps mur) que runtime.
 - native : ~13x moins de CPU (~100% mono-coeur vs ~257% JVM JIT+GC).
 - native : 3.3x moins de RAM (RSS pic).
 - native : demarrage 5.4x plus court.
 - prix : executable autonome 37 MB (vs jar + JVM) et build natif plus long.

Détails par métrique

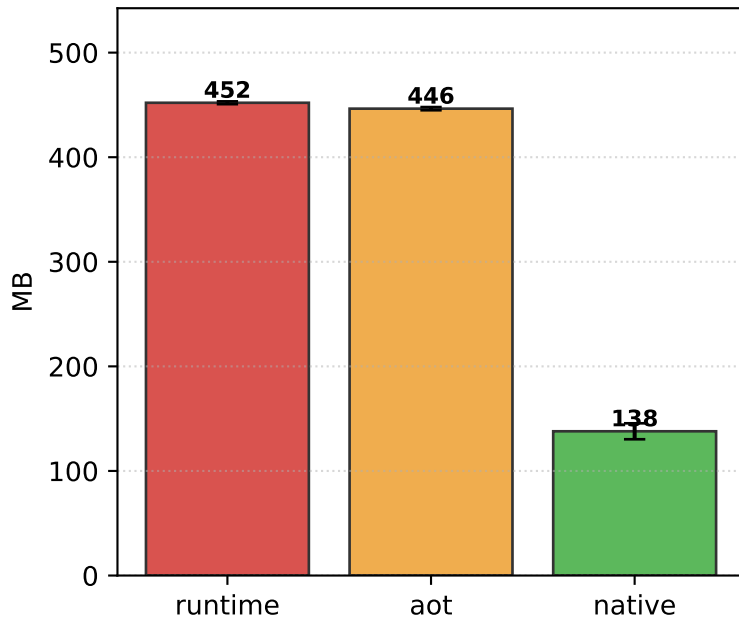
Temps mur total (moy $\pm$ σ)



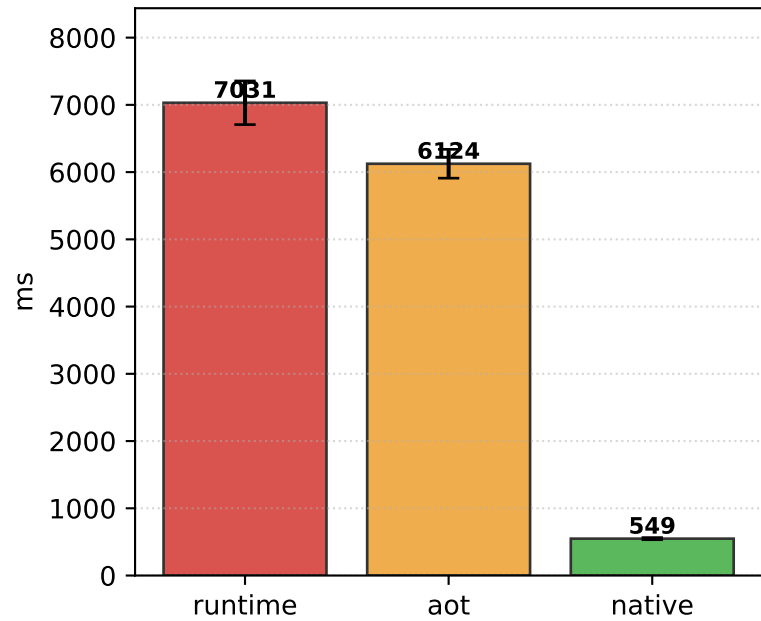
Démarrage applicatif



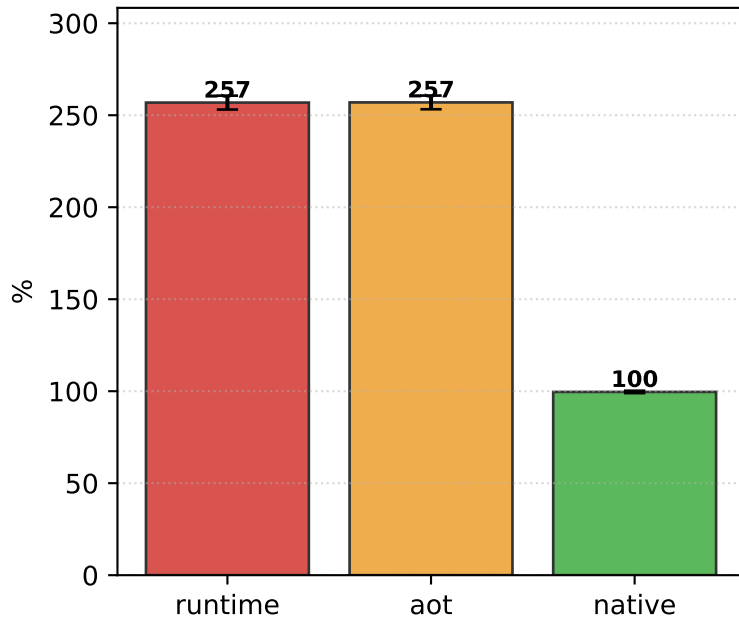
Mémoire — RSS pic



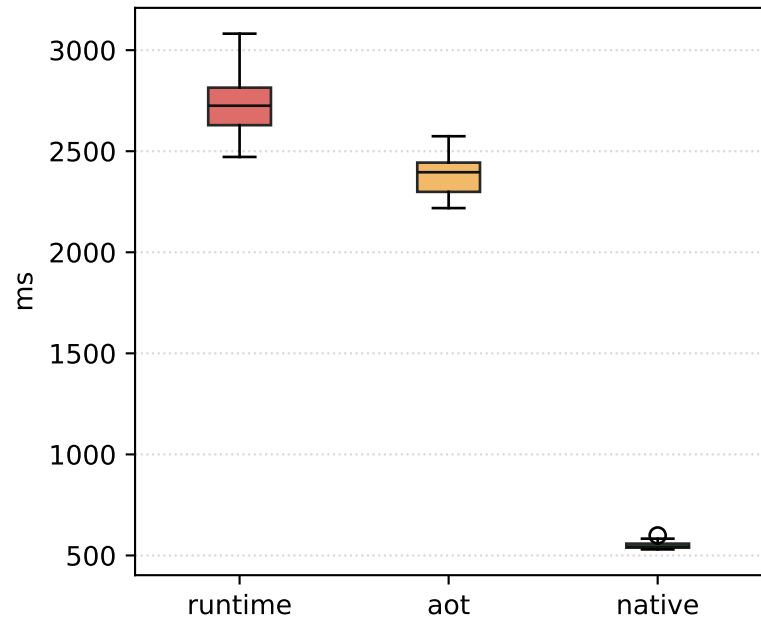
CPU consommé (user+sys)



Occupation CPU moyenne

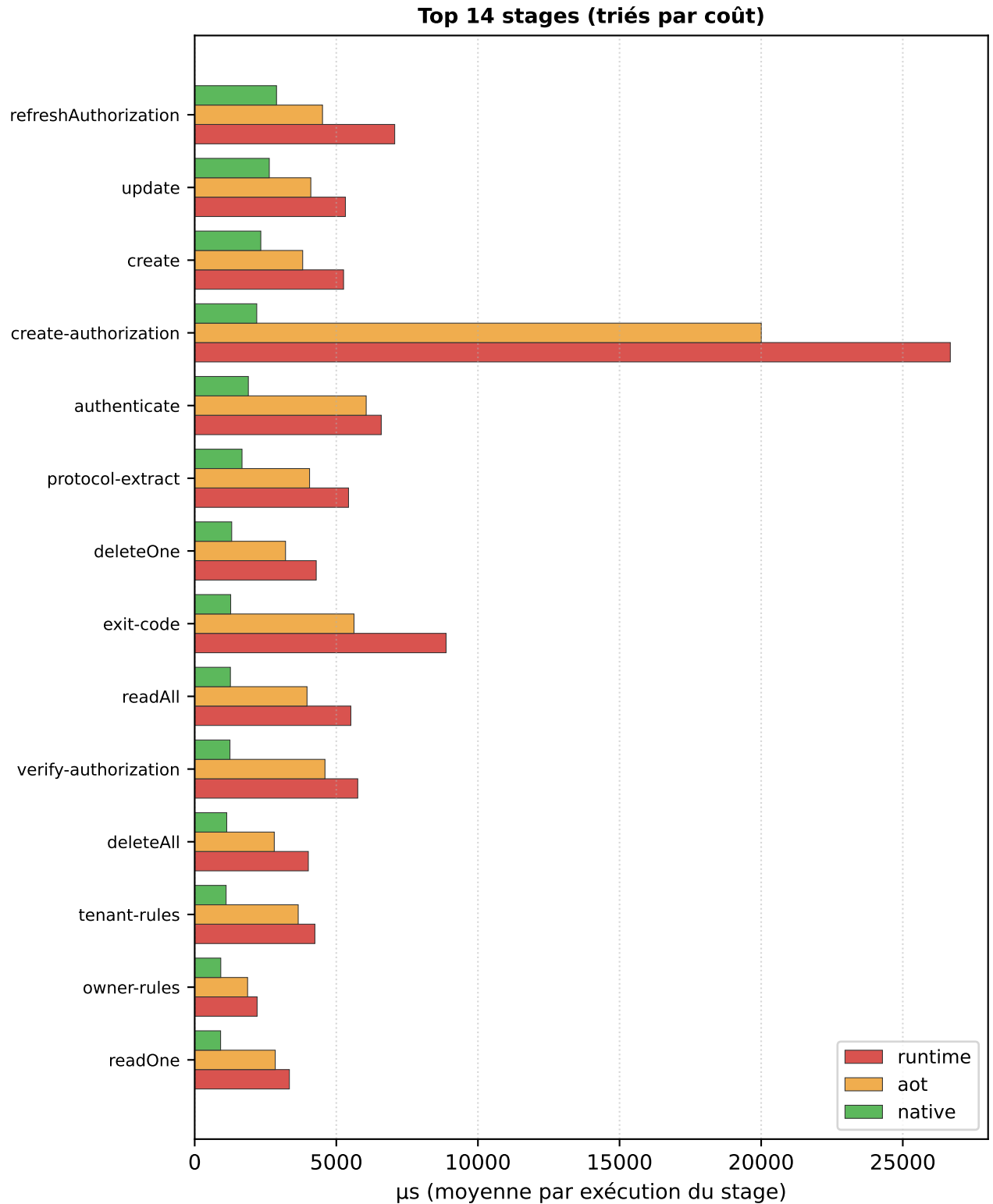


Distribution du temps mur



Timings du pipeline — temps moyen par stage

Durée moyenne (μ s) de chaque stage du workflow, par mode (1 run représentatif, agrégé sur les opérations).



Lecture : les stages cryptographiques (create-authorization : génération de clé EC + signature) dominant. Chaque stage est plus rapide en natif ; sur un service long les modes JVM se rapprochent au chaud mais paient toujours startup, RAM et CPU de warmup.

Statistiques d'exécution — opérations API

Par opération : nombre d'appels, succès/échecs, temps moyen (µs) par mode.

opération	appels	ok/ko	runtime µs	aot µs	native µs
tenants:tenants-create-one-tenant	1	1/0	155982	125772	18821
tenants:tenants-delete-all-tenants	1	1/0	55108	51988	17690
tenants:tenants-delete-one-tenant	1	1/0	51675	49728	16042
tenants:tenants-read-all-tenants	2	2/0	79234	58982	21501
tenants:tenants-read-one-tenant	2	2/0	93660	51079	16575
tenants:tenants-update-one-tenant	1	1/0	79923	54899	15938
users:users-create-one-user	4	4/0	105210	84954	29068

Tous les stages — temps moyen (µs) par mode

stage	appels	runtime µs	aot µs	native µs
refreshAuthorization	8	7062	4513	2890
update	24	5322	4100	2634
create	24	5256	3813	2334
create-authorization	8	26678	20004	2191
authenticate	8	6587	6055	1895
protocol-extract	24	5432	4056	1671
deleteOne	24	4291	3207	1307
exit-code	24	8876	5625	1271
readAll	24	5514	3966	1261
verify-authorization	24	5757	4600	1245
deleteAll	24	4012	2810	1132
tenant-rules	24	4244	3655	1109
owner-rules	8	2204	1868	921
readOne	24	3343	2844	915
deserialize	24	3488	3294	780
verify-owner	8	4243	2837	748
verify-authority	24	3708	3006	740
verify-tenant	24	3718	3050	703
protocol-response	24	1978	1558	395
serialize	24	2130	1613	324
init-codes	24	1725	1289	150